浏览器自动化：从GUI到OpenCLI

明径 大淘宝技术 2026年5月22日 02:13 浙江 17人



文章讲述放弃不稳定的前端UI自动化操作，采用解析并复现底层API请求的方式，来解决浏览器自动化的效率与稳定性难题。（文章内容基于作者个人技术实践与独立思考，旨在分享经验，仅代表个人观点。）

01 为什么我们需要浏览器自动化

如今大量业务系统都跑在浏览器里——运营配置后台、工单处理系统、发布运维平台。如果能让这些系统自动运转，对提效和智能化运营的价值不言而喻。

但现实是，Agent 想操控浏览器，路并不好走。

02 现有方案的困境

	方案	实现原理	问题
MCP	平台本身提供MCP	agent通过MCP协议操作浏览器功能	• 大多数内部系统根本没为自动化设计过接口 • 即便有，往往也只覆盖核心链路，边缘功能全靠前端直连后端 • 开发 MCP Server 要处理鉴权、版本兼容、部署维护，成本不低
GUI	Puppeteer	CDP 协议封装	• 脆弱：界面改个按钮位置、弹窗慢半秒出来，整个流程就断了 • 难组合：截图→识别→点击→等待，每一步都是黑盒，串起来像搭积木，风一吹就倒 • 贵且慢：每步都要调多模态模型，Token 烧得快，响应还慢
	Playwright	CDP + 自研协议	
	browser-use	封装 Playwright/Puppeteer + AI 推理	
	agent-browser	封装 Playwright + Agent 集成	
CLI	CLI-Anything	分析——扫描源码、规划命令分组、状态模型、输出格式、构建 Click CLI	依赖源码和平台本身开放
	OpenCLI	抓包+复用浏览器 cookie	✅



核心想法很简单：不跟网页界面较劲，直接抓它背后的 API。

浏览器里看到的数据，本质上都是前端从某个接口拿回来的。把这个接口找出来、把请求复现出来，比点按钮靠谱得多。

快速上手

```
npm install -g @jackwener/opencli
```

直接使用：

```
opencli list
# 查看所有命令
opencli list -f yaml
# 以 YAML 列出所有命令
```

```
opencli hackernews top --limit
5
# 公共 API，无需浏览器
opencli bilibili hot --limit
5
# 浏览器命令
opencli zhihu hot -f json
# JSON 输出
opencli zhihu hot -f yaml
# YAML 输出
```

原理分析

AI Agent 探索工作流

步骤	工具	做什么
0. 打开浏览器	`browser_navigate`	导航到目标页面
1. 观察页面	`browser_snapshot`	观察可交互元素（按钮/标签/链接）
2. 首次抓包	`browser_network_requests`	筛选 JSON API 端点，记录 URL patter n
3. 模拟交互	`browser_click` + `browser_w ait_for`	点击"字幕""评论""关注"等按钮
4. 二次抓包	`browser_network_requests`	对比步骤 2，找出新触发的 API
5. 验证 AP I	`browser_evaluate`	`fetch(url, {credentials:'include'})` 测试返回结构
6. 写代码	—	基于确认的 API 写适配器

懒加载机制

```
> [!CAUTION]
> **你（AI Agent）必须通过浏览器打开目标网站去探索！**
```

```
> 不要只靠 `opencli explore` 命令或静态分析来发现 API。
> 你拥有浏览器工具，必须主动用它们浏览网页、观察网络请求、模拟用户交互。

### 为什么？

很多 API 是**懒加载**的（用户必须点击某个按钮/标签才会触发网络请求）。字幕、评论、关
```

五级认证策略

OpenCLI 提供 5 级认证策略。使用 `cascade` 命令自动探测：

```
opencli cascade https://api.example.com/hot
```

策略决策树：

```
直接 fetch(url) 能拿到数据?
  → ✅ Tier 1: public（公开 API，不需要浏览器）
  → ❌ fetch(url, {credentials:'include'}) 带 Cookie 能拿到?
      → ✅ Tier 2: cookie（最常见，evaluate 步骤内 fetch）
      → ❌ → 加上 Bearer / CSRF header 后能拿到?
            → ✅ Tier 3: header（如 Twitter ct0 + Bearer）
            → ❌ → 网站有 Pinia/Vuex Store?
                  → ✅ Tier 4: intercept（Store Action + XHR 拦截）
                  → ❌ Tier 5: ui（UI 自动化，最后手段）
```

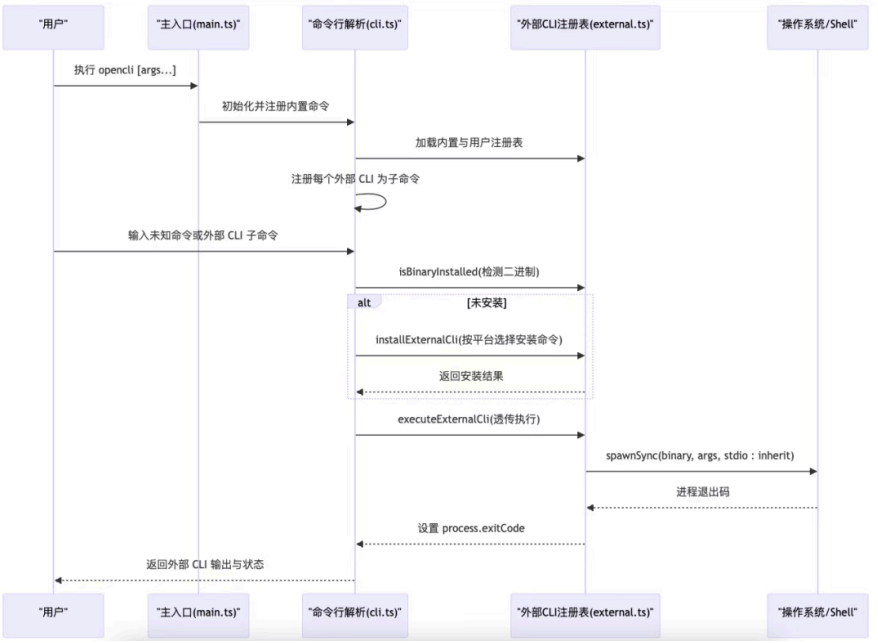
适配器

```
你的 pipeline 里有 evaluate 步骤（内嵌 JS 代码）?
  → ✅ 用 TypeScript（src/clis/<site>/<name>.ts），保存即自动动态注册
  → ❌ 纯声明式（navigate + tap + map + limit）?
      → ✅ 用 YAML（src/clis/<site>/<name>.yaml），保存即自动注册
```

场景	选择	示例
纯 fetch/select/map/limit	YAML	`v2ex/hot.yaml`，`hackernews/top.yaml`
navigate + evaluate(fetch) + map	YAML（评估复杂度）	`zhihu/hot.yaml`
navigate + tap + map	YAML ✅	`xiaohongshu/feed.yaml`，`xiaohongshu/notifications.yaml`
有复杂 JS 逻辑（Pinia state 读取、条件分支）	TS	`xiaohongshu/me.ts`，`bilibili/me.ts`
XHR 拦截 + 签名	TS	`xiaohongshu/search.ts`
GraphQL / 分页 / Wbi 签名	TS	`bilibili/search.ts`，`twitter/search.ts`

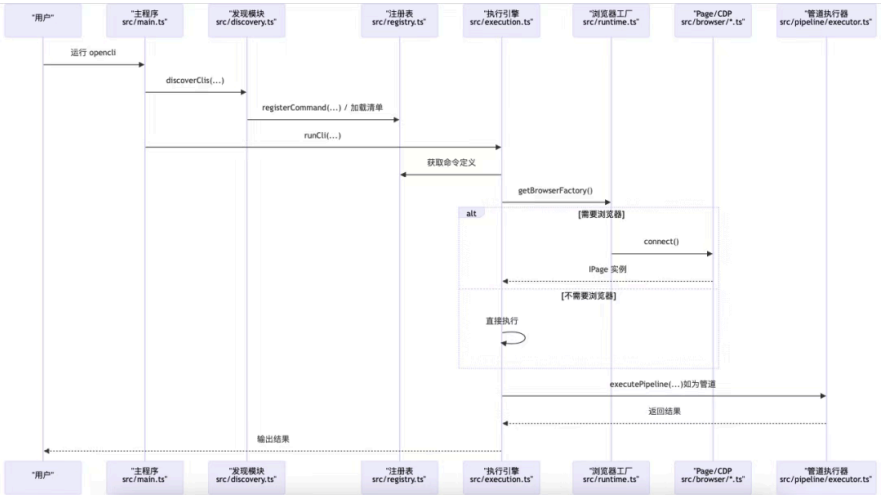
外部CLI集成

也支持现有CLI直接集成到OpenCLI



CLI执行流程

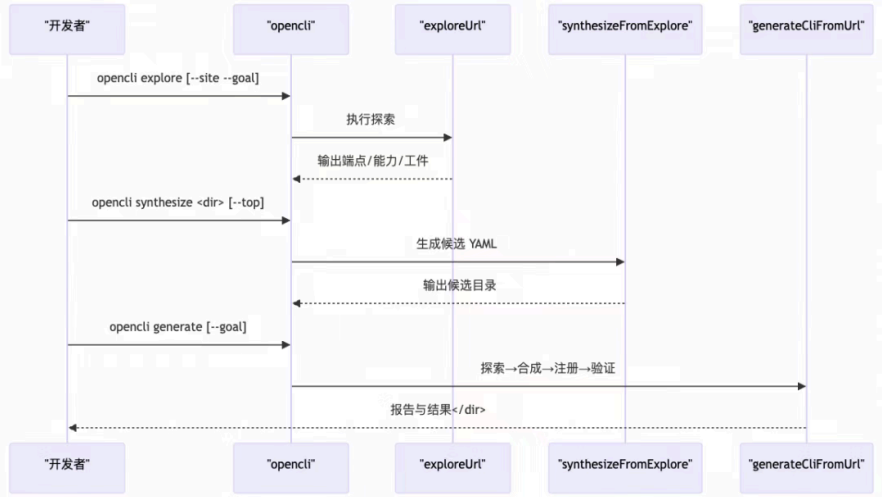
下图展示从启动到执行的关键路径：入口加载命令清单，构建注册表；执行阶段根据策略与浏览器需求选择适配器或管道步骤，完成数据采集与输出。



自动生成CLI

AI 原生生成CLI流程

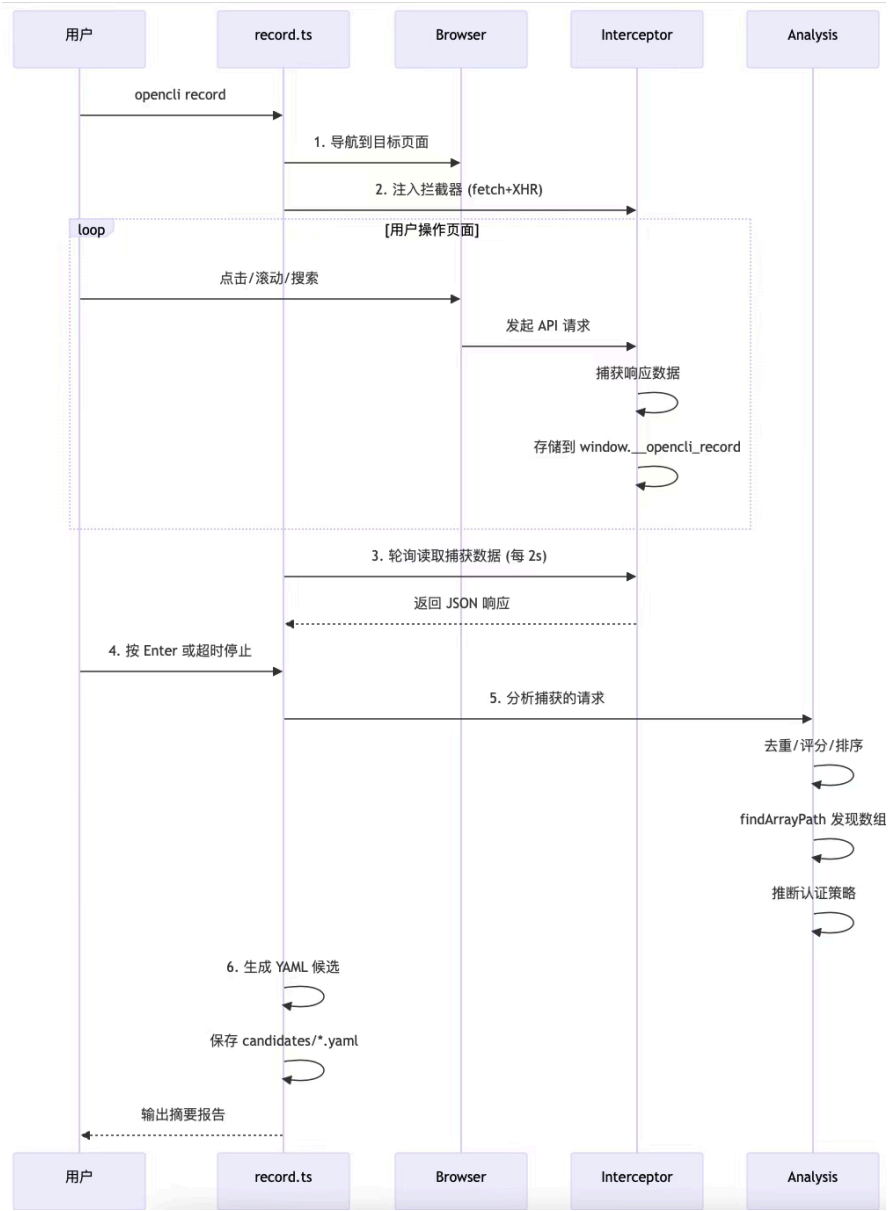
1. 探索与分析：explore 深度抓取页面、自动滚动、拦截网络请求、识别框架与状态管理、推断能力与推荐参数。
2. 策略选择：根据鉴权头/签名等特征自动选择策略（public/cookie/header/intercept/store-action）。
3. 适配器合成：synthesize 基于探索产物生成候选 YAML，自动模板化 URL、字段映射与参数默认值。
4. 测试与验证：generate 串联探索→合成→注册→验证，支持目标化选择与回退策略。



Record操作录制

opencli record 采用“浏览器录制 – 智能回放”模式：启动浏览器后，捕获用户在目标 URL 上的交互行为及产生的网络请求。系统通过对请求序列进行评分排序与语义分析，自动生成可复用的 CLI 命令。

执行流程如下图所示：



当前局限性：

- 请求体（Payload）缺失：目前的录制引擎仅捕获请求元数据（url, method, body: responseBody），未能完整提取 POST/PUT 等写操作中的 Request Body。
- 生成能力受限：由于缺乏关键参数载荷，自动化脚本生成逻辑目前仅能覆盖只读类接口（如列表查询、详情获取并输出 YAML），无法有效支撑写操作类接口（如创建、更新、删除）的命令生成，导致自动化闭环在“写入场景”中断。

QoderWork自动生成CLI

为了方便自动生成CLI命令，我整理了如下的Skill，其中CLI-ONESHOT.md和CLI-EXPLORER.md可在开源项目中自行下载。

SKILL.md

```
---
name: opencli
description: "Generate CLI adapter files (YAML/TypeScript) for the ope
---

# OpenCLI Adapter Generator

## Overview

OpenCLI is a CLI framework that wraps website APIs into local command-

## Workflow Modes

**Quick mode** (single command): Follow [CLI-ONESHOT.md](./references/

**Full mode** (complex adapters): Read [CLI-EXPLORER.md](./references/

## Output Specification

All adapter files **must** be written to `~/.opencli/clis/{site}/{comm

Correct examples:
- `~/.opencli/clis/aem/page-views.ts`
- `~/.opencli/clis/twitter/lists.yaml`
- `~/.opencli/clis/bilibili/favorites.ts`

## Supported Formats

| Format | Extension | When to use |
|--------|-----------|-------------|
| YAML | `.yaml` | Simple scenarios (Cookie/Public auth, straightforwa
| TypeScript | `.ts` | Complex scenarios (Intercept capture, Header au

## Standard Workflow

1. **Create directory**: `mkdir -p ~/.opencli/clis/{site}`
2. **Generate adapter file** at the correct path (YAML or TS)
3. **Verify**: `opencli list | grep {site}` then `opencli {site} {comm

## Naming Conventions

| Element | Rule | Good | Bad |
|---------|------|------|-----|
| site | Lowercase, hyphens allowed | `aem`, `my-site` | `AEM`, `my_si
```

```
| command | Lowercase, hyphen-separated | `page-views`, `project-info`

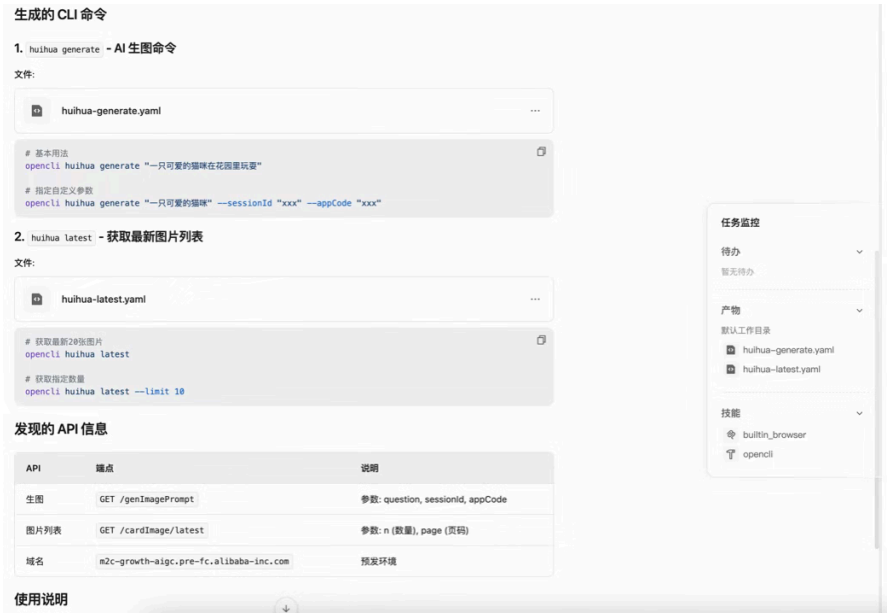
## Pre-Generation Checklist

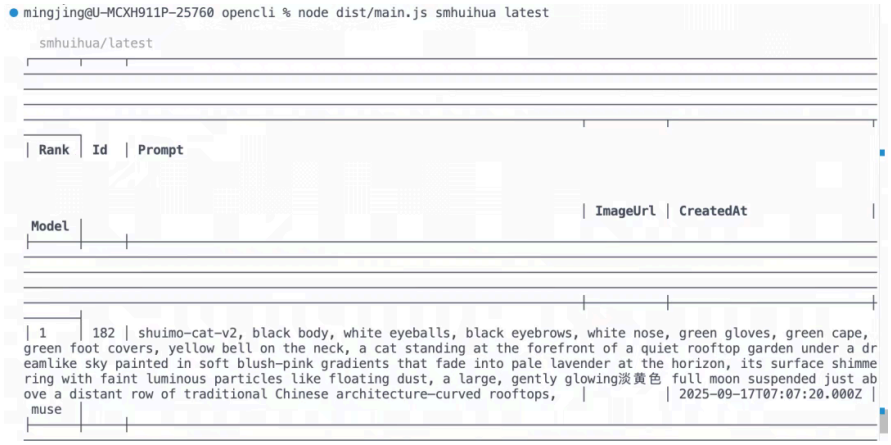
- [ ] Output path is `~/.opencli/clis/{site}/{command}.yaml` or `.ts`
- [ ] Site name is lowercase (no uppercase, no underscores)
- [ ] Command name uses hyphens (no spaces, no underscores)
- [ ] File extension is `.yaml` or `.ts` only
- [ ] Directory `~/.opencli/clis/{site}/` has been created
```



使用case：

内部会画平台CLI化



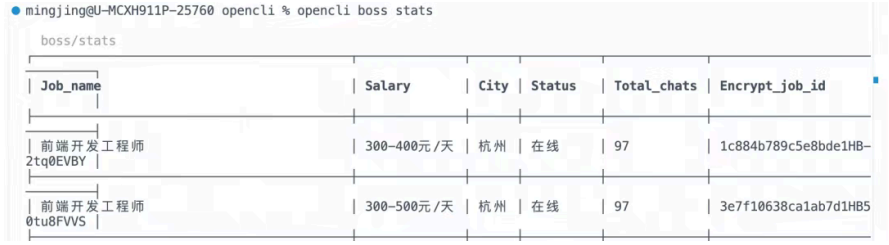


BOSS招聘自动化案例展示

1.帮我和候选人沟通



2.统计招聘数据



04 未来软件竞争维度：从界面到可调用性

未来的软件，不会只服务人，也会服务 Agent。

以前我们评价一个 SaaS，看的是界面顺不顺、按钮好不好点。但 Agent 不会欣赏你的按钮做得多圆。它只在乎一件事：能不能稳定调用你。

GUI 是给人用的。API 是能力底座。而 Agent 最喜欢的，其实是更清晰的执行面：命令、参数、返回值、失败原因。

未来软件可能会多一个新竞争维度：不是谁页面更好看。而是谁更容易被 Agent 理解、调用、验证，再接进工作流。唯有如此，才更有机会成为下一代工作流里的基础节点。

过去的软件竞争界面，未来的软件竞争可调用性。

¤ 拓展阅读 ¤

3DXR技术 | 终端技术 | 音视频技术

服务端技术 | 技术质量 | 数据算法



大淘宝技术

大淘宝技术官方账号

967篇原创内容

公众号

阅读 6955